



Processor: A hardware component that automatically executes a set of instructions.

Thread: A thread is a small unit of runs inside a process, and it has its own context (data).

- It has its own **context (data)**
- It can stop and continue later

Process: A process is a program that runs on a computer and contains one or more threads.

- Threads run inside a process

Processor context:

The processor context is the **minimum set of values stored in processor's registers** that are needed to execute series of instructions.

It includes values such as:

- program counter
- stack pointer
- registers

Thread context:

The thread context is the **minimum set of values stored in registers and memory**, used for execute a series of instructions.

It includes:

- processor context
- thread state

Process context:

The process context is the **minimum set of values stored in registers and memory**, used to execute a thread, including Memory Management Unit (MMU) register values.

It includes:

- thread context
- memory management data (MMU registers)

Main idea:

- Thread context = data for **one thread**
- Process context = data for **the whole process (includes threads)**

Context switching – Observations:

- Threads share the same **memory (address space)** , Thread switching can be done **without the OS**.
- Process switching is **more expensive** ,It needs help from the **operating system**
- Creating and deleting threads is **cheaper** than processes



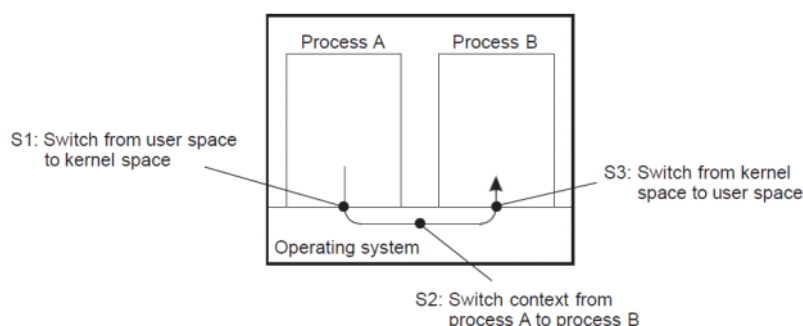
Why do we use threads?

- **Avoid blocking:**
If one thread waits (for I/O), another thread can run
- **Use parallelism:**
Threads can run at the same time on multiple cores
- **Avoid process switching:**
Using threads is easier and faster than using many processes

Threads improve **performance** by reducing waiting time, enabling parallel execution, and avoiding expensive process switching.

Avoid process switching:

Avoid expensive context switching



- Process switching is **expensive** Because ;
 - It needs switching between **user space and kernel space**
 - The OS must change from one process to another
- 👉 This takes **time and resources**

Trade-offs:

- Threads share the same address space at memory. Thus it allows context switching to happen independently of the OS → **more errors possible**
- No strong protection between threads
- But Thread context switching is **faster**

Threads help:

- Threads can switch **faster** than processes
- No need for full process switching

Threads and operating systems:

Main issue:

- Should threads be managed by the **OS (kernel)** or should they be implemented as **user-level packages**?



User-space solution:

- All thread operations are handled inside **one process** → **very efficient**
- The OS sees only the **process**, not individual threads
→ if one thread is blocked by the kernel, the **whole process is blocked**
- Hard to manage events → OS **cannot easily control** or signal individual threads

User-space solution means threads are implemented at user level using libraries, not by the OS kernel

Kernel solution:

- Threads are managed by the **operating system (kernel)** by using **system calls**.
- All operations use **system calls**

Advantages:

- If one thread blocks, the **kernel runs another thread**
- The kernel can easily handle **events**

Disadvantage:

- Every operation needs a **kernel call** → this can **reduce performance**

Conclusion:

- Mixing user-level and kernel-level threads is complex
- The performance gain is **not very high**

👉 Main idea (exam):

Kernel threads **give better control**, but they are **slower** due to system call overhead.

Using threads at the client side:**Multithreaded web client:**

- A browser finds many files in a page
- Each file is downloaded by a **separate thread**
- The browser shows files as they arrive

Multiple requests (RPC):

- A client sends many requests at the same time
- Each request uses a **different thread**
- The client waits for all results
- If requests go to different servers → **faster execution**

parallel request

Using threads at the server side:**Improve performance:**

- Creating a thread is **cheaper than a process**
- Threads help use **multiple processors** → improve **performance**
- Servers can handle new requests while user waiting → **hide network latency**

multi-user
handling

Better structure:

- Servers often use **I/O operations**
- Threads make programs **simpler and easier to understand**

Dispatcher/Worker model:

- A **dispatcher thread** receives requests
- It sends each request to a **worker thread**
- Worker threads execute requests in **parallel**

System Models Characteristics

- ! ? **Multithreading:** A model characterized by **parallelism** and the use of blocking system calls.
- ? **Single-threaded process:** A model characterized by no parallelism and the use of blocking system calls.
- ? **Finite-state machine:** A model characterized by parallelism and the use of **nonblocking** system calls.

Model	Characteristics
Multithreading	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls

Virtualization:

Why it is important:

- !
- Hardware changes **faster than software**
 - Makes programs **easy to move (portability) and code migration**
 - Provides **isolation** of failing and attacks

Principle:

- **Virtualization mimics interfaces** - > A program can run on different systems using the same interface

Mimicking interfaces:

Virtualization works by **copying (mimicking) interfaces** so programs can run on different systems.

Three main interface types:

1. **Instruction Set Architecture (ISA):** the set of machine instructions, with two subsets:

- Two types:
 - **Privileged instructions:** Instructions allowed to be executed only by the operating system.
 - **General instructions:** Instructions that can be executed by any program.

2. **System calls :** Functions provided by the **operating system**

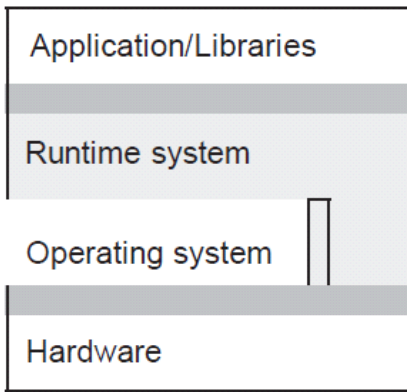
3. **Library calls (API):** known as **API** . Functions used by programmers

Ways of Virtualization

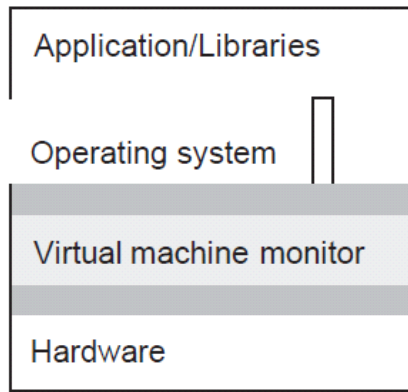
Process VM: Runs on top of an operating system and uses an interpreter.

Native VMM: Runs directly on hardware with a small OS.

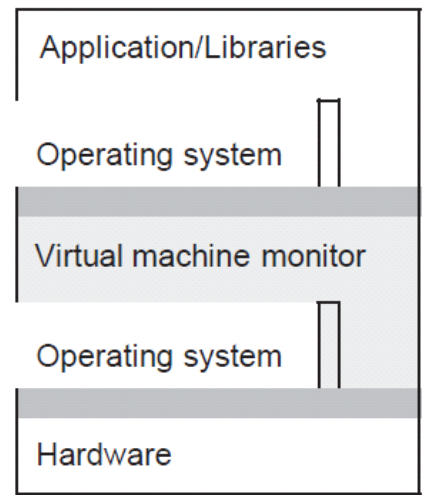
Hosted VMM: Runs on top of an operating system and uses it.



(a)



(b)



(c)

Zooming into Virtual Machines (VMs)

Virtual Machine Organization: A layered system structure that stacks Application/Libraries, Guest operating system, Virtual machine monitor, Host operating system, and Hardware.

- **Privileged instruction:**
An instruction that **causes a trap to the OS** if it is run in **user mode**. ONLY OS can run it.
- **Nonprivileged instruction:**
Any remaining standard instruction that **does not trigger an operating system trap**.
- **Control-sensitive instruction:**
A special instruction that can **change system settings** (like registers or interrupt table).
- **Behavior-sensitive instruction:**
A special instruction whose result **depends on the context** (where and how it runs).

VMs and Cloud Computing

3 types of cloud servicing ;

- **IaaS (Infrastructure-as-a-Service):** A cloud service model covering the basic computing infrastructure.
- **PaaS (Platform-as-a-Service):** A cloud service model covering system-level services.
- **SaaS (Software-as-a-Service):** A cloud service model containing actual ready-to-use applications

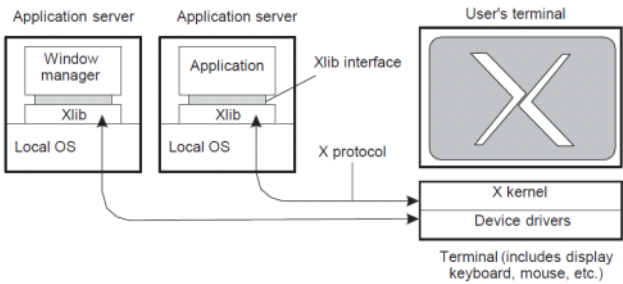
at IaaS:

- **VM Rental:**
Cloud providers give users a **virtual machine** instead of a physical one.
This VM **shares hardware with other users**.
- **Isolation:**
Users are **separated from each other** for security. But performance may **not always be fully separated**.

Client-server interaction

Example: The X Window system

Basic organization



In the X Window System, the application acts as a client, and the X system acts as a server that handles display and input on the user's machine.

Improving X: Problems:

- No clear separation between **application logic and UI**
- Applications work in a **synchronous way** with the X system

Alternative approaches:

- Let applications control the display **completely** (e.g., VNC)
- Or use **high-level display commands** (like draw window, draw button)

Clients and Transparency

- **Client-side software:** Software generally tailored to achieve distribution transparency.
- **Access transparency:** Utilizes client-side stubs for remote procedure calls.
- **Location/migration transparency:** Allows client-side software to keep track of a server's actual location.
- **Replication transparency:** Handles multiple server invocations directly within the client stub.
- **Failure transparency:** Placed at the client to mask server and communication failures.

Servers

Server Types

- ! **Iterative server:** Server handles the request before attending a next request. tek tek
- Concurrent server:** A server uses a dispatcher, it picks up incoming requests and passes them to separate threads or processes.

Observation: most services are tied to a specific port

ftp-data	20	File Transfer [Default Data]
ftp	21	File Transfer [Control]
telnet	23	Telnet
smtp	25	Simple Mail Transfer
www	80	Web (HTTP)

Contacting a server:

- Most services use a **specific port number** (e.g., HTTP → 80, FTP → 21)

Dynamic endpoint assignment:

- The client first **asks for a service**
- The server (or daemon) gives an **endpoint (port)**
- Then communication continues using that endpoint
- A **super-server** can create a server and handle the request

Dynamically assigning an end point



Out-of-band Communication (Interrupting a Server)

Is it possible to interrupt a server once it has accepted (or is in the process of accepting) a service request?

- ! 1. **Use a separate port for urgent data**, Server has a separate thread/process for urgent messages. It requires an OS that supports priority-based scheduling.
- 2. **Use transport layer facilities**, such as catching **TCP urgent messages** using operating system signaling techniques.

Server State



- **Stateless servers:** Servers that never keep accurate information about the status of a client after having handled a request.

Characteristics:

- Do not remember opened files
- Do not manage client cache
- Do not keep client information

Consequences:

- Client and server are **independent**
- Less problems after crashes
- May reduce performance

- **Stateful servers:** Servers that keep track of the status of their clients, such as recording open files or tracking cached data.

Observation:

- Can have **very high performance**
- Clients can store data locally
- Reliability is usually **not a big problem**

Server Clusters

Three Different Tiers

Three-tier architecture:

- Systems are divided into **three layers (tiers)**

1. First tier:

- Receives client requests
- Sends requests to the correct server (**request dispatching**)

2. Second tier:

- Application servers
- Process the requests

3. Third tier:

- Databases or file systems
- Store and manage data

Request Handling

What is the problem?

- The **first tier** handles all requests
- This can create a **bottleneck** (too much load on one place) - System becomes slow



TCP handoff : means moving a client request from one server to another server.
This helps **reduce bottlenecks** and **improve performance**.

Problem



- The **front-end server** can get overloaded (too many requests)

Transport-layer switching: The front end passes the TCP request to a server based on specific performance metrics.

Content-aware distribution: The front end **reads the request** content to select the best server. Smarter but slower

When servers are spread across the Internet

Observation

- Servers are in different locations on the Internet
- This can cause **management problems**

Solution: DNS-based request dispatching

The client asks DNS for a service, and DNS knows many servers.

DNS returns the closest server, so the client connects to it.

- The client does not know the system is distributed because DNS works for it. But DNS may not be close to the client, so the result may not be perfect.

Reasons to migrate code

Load distribution

- Servers should be used properly to **avoid wasting energy**
- Computations should be **close to the data** to **reduce communication**

Flexibility : moving code to a client when needed

Result :

- No need to install software before
- System becomes more flexible and dynamic

Models for Code Migration

- **CS (Client-Server):** A model where the server holds the code and resources, and performs the execution on behalf of the client. (code is already on server)
- **REV (Remote Evaluation):** A model where the **client provides the code** to the server, and the server performs the execution using its own resources. (code moves to server before execution)
- **CoD (Code-on-demand):** A model where the client retrieves the code from the server and executes it locally using its own resources. (**only code moves to client**)
- **MA (Mobile Agents):** A model where the client migrates both the code and the execution state to the

server to continue execution using the server's resources. (**code + state + resources move together**)

Strong and Weak Mobility

Object components

- Code segment: contains the code
- Data segment: contains the data (state)
- Execution state: contains the running state of the program

Mobility Types

- **Weak mobility:** Moving only the code and data segments of a component and rebooting execution. Two types : code shipping(push) from code fetching(pull)
- **Strong mobility:** Moving the component , including execution state.
 - Migration: move whole object to another machine
 - Cloning: create a copy with same state

Migration in heterogeneous systems

- Target machine may not run the code
- Execution depends on local hardware and operating system

Solution

- Use an abstract machine on all platforms

Examples

- Interpreted languages use a virtual machine
- Virtual machine monitors (VMM) help run code on different systems

Migrating a Virtual Machine

1. Push memory pages to the new machine and resend any pages modified during the migration process.
2. Stop the current virtual machine, migrate the memory, and start the new virtual machine.
3. Start processes on the new virtual machine immediately and pull memory pages in on demand.